Cold Storage Monitoring System

with FastAPI, Role-Based Access Control, and Automated Sensor Simulation

---

Academic Project Report

Submitted in partial fulfillment of the requirements for
the completion of the software project / internship project

Submitted By: _____

Registration No.: _____

Department: _____

Institution: _____

Guide / Mentor: _____

Technology Stack: Python, FastAPI, SQLAlchemy, SQLite, HTML, CSS, Jinja2

Date: June 10, 2026

Certificate / Declaration

This is to certify that the project report entitled **"Cold Storage Monitoring System with FastAPI, Role-Based Access Control, and Automated Sensor Simulation"** is a record of project work carried out by the undersigned student as part of the academic/project submission requirements.

The project has been developed as a web-based application for monitoring cold storage environments using automated sensor simulation, role-based access control, warehouse management, sensor data processing, alert generation, and dashboard analytics. The work described in this report is original to the best of my knowledge and has not been submitted elsewhere for the award of any degree, diploma, or certification.

Student Signature	**Guide / Mentor Signature**
_____	_____
Name: _____	Name: _____
Date: _____	Date: _____

Acknowledgement

I would like to express my sincere gratitude to my project guide, mentors, faculty members, and all those who supported me during the development of this project. Their guidance, technical suggestions, and encouragement helped me understand the practical aspects of software development, backend engineering, database design, and web application deployment.

I am also thankful to my institution for providing the opportunity to work on a practical software project that combines modern web technologies with a real-world problem domain. The development of the Cold Storage Monitoring System helped me gain hands-on experience in FastAPI, SQLAlchemy, role-based authentication, database modeling, automated sensor simulation, and dashboard-based monitoring.

Finally, I thank my family and friends for their support and motivation throughout the project development and documentation process.

Abstract

Cold storage environments are essential for preserving temperature-sensitive products such as medicines, vaccines, biological samples, and pharmaceutical inventory. Any abnormal change in temperature, humidity, or door status can compromise product quality, cause financial loss, and create compliance risks. Traditional manual monitoring methods are often slow, error-prone, and insufficient for continuous monitoring.

This project presents a web-based **Cold Storage Monitoring System** developed using **FastAPI**, **SQLAlchemy**, **SQLite**, **HTML**, **CSS**, and **Jinja2 templates**. The system allows administrators to manage users and assign warehouses, while users can create storage units, view automatically generated sensors, monitor sensor readings, and resolve alerts related to their own storage units. The system implements role-based access control with two major roles: Admin and User.

A key feature of the system is its automated simulator module. The simulator runs along with the FastAPI backend, fetches active sensors, randomly selects sensors, generates realistic and abnormal readings, and submits those readings to the backend data endpoint. Based on rule conditions such as high temperature, high humidity, and open door status, the system generates alerts and stores them in the database. These alerts are displayed on dashboards with visual analytics such as pie charts and bar graphs.

The project demonstrates a complete workflow involving user registration, authentication, warehouse assignment, storage unit creation, automatic default sensor creation, sensor data generation, alert detection, dashboard visualization, and alert resolution. The final system provides a practical foundation for real-world cold storage monitoring and can be extended to integrate physical IoT sensors, cloud deployment, notification services, and predictive analytics.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Background

Cold storage systems play an important role in pharmaceutical logistics, healthcare supply chains, food preservation, and laboratory environments. In pharmaceutical storage, products such as vaccines, insulin, diagnostic kits, biological samples, and temperature-sensitive medicines must be stored within strict environmental limits. If the storage temperature or humidity exceeds safe thresholds, the product quality may degrade, resulting in financial loss and possible health risks.

Modern cold storage facilities require continuous monitoring of environmental parameters. Manual inspection is not sufficient because environmental conditions can change at any time. A door may remain open accidentally, a cooling unit may fail, or humidity levels may rise unexpectedly. Such events must be detected immediately so that corrective actions can be taken.

The Cold Storage Monitoring System developed in this project addresses this requirement by providing a web-based platform for monitoring warehouses, storage units, sensors, sensor-generated data, and alerts. The system uses FastAPI for backend services, SQLAlchemy for database operations, SQLite for local storage, and Jinja2 templates for server-side rendered web pages.

1.2 Problem Statement

Cold storage facilities require reliable monitoring of temperature, humidity, and door status. In many small or medium-scale environments, monitoring may still depend on manual checks or disconnected devices. These approaches suffer from delayed detection, lack of centralized data, limited reporting, and poor accountability.

The major problem addressed by this project is the absence of a centralized, role-based, automated monitoring system that can simulate sensor readings, detect abnormal conditions, generate alerts, and provide meaningful dashboard analytics for both administra-

tors and users.

Therefore, the project aims to develop a complete software system where administrators can manage users and warehouses, users can manage assigned storage units and sensors, and the simulator can automatically generate readings to test alert conditions.

1.3 Need for the System

The need for this system arises from the following practical requirements:

- Continuous monitoring of cold storage conditions.
- Early detection of unsafe temperature or humidity values.
- Detection of open door status that may affect internal temperature.
- Role-based separation between administrator and user operations.
- Automatic creation of sensors for each storage unit.
- Automatic generation of sensor data for testing and demonstration.
- Dashboard analytics for alert trends and storage unit conditions.
- Historical record keeping for future audit and analysis.

A web-based system is suitable because it can be accessed easily through a browser and can provide centralized data visibility. The use of APIs also allows future integration with actual IoT devices or external monitoring systems.

1.4 Objectives

The main objectives of the Cold Storage Monitoring System are:

1. To develop a web-based application for cold storage monitoring.
2. To implement role-based access control for Admin and User roles.
3. To allow users to register and log in securely.
4. To allow administrators to view users and assign warehouses.
5. To allow users to manage assigned warehouses and storage units.

6. To automatically create default sensors when a storage unit is created.
7. To simulate sensor readings for temperature, humidity, and door status.
8. To generate alerts when readings violate predefined rules.
9. To display alert analytics using pie charts and bar graphs.
10. To allow users to resolve only their own alerts.
11. To maintain database records for users, warehouses, sensors, data, alerts, rules, and alert logs.

1.5 Scope of the Project

The scope of this project includes backend development, frontend page rendering, database design, role-based login, warehouse assignment, storage unit management, automatic sensor creation, sensor data simulation, alert generation, alert resolution, and dashboard visualization.

The project is primarily designed as an academic and demonstration-level monitoring system. It uses simulated sensor data instead of physical IoT hardware. However, the architecture is designed in such a way that physical sensors can be integrated in the future by sending readings to the same backend data endpoint.

The project scope includes:

- Admin dashboard and user dashboard.
- User registration and login.
- Warehouse assignment by admin.
- Storage unit creation by users.
- Automatic creation of temperature, humidity, and door status sensors.
- Sensor data simulation and backend data submission.
- Rule-based alert generation.
- Alert filtering based on user ownership.
- Alert visualization through charts.
- Basic dependency cleanup during deletion operations.

The project does not include real hardware integration, SMS/email notifications, cloud deployment, or machine learning prediction in the current version, but these are considered future enhancements.

2 Literature Survey / Existing System

2.1 Existing Methods in Cold Storage Monitoring

Traditional cold storage monitoring systems often depend on manual temperature logs, standalone thermometers, or local display devices attached to refrigeration units. In such systems, employees periodically check the temperature and humidity and record the values in registers or spreadsheets. Although this approach is simple, it is not reliable for critical environments where changes can occur between inspection intervals.

Some facilities use standalone data loggers that record temperature values. These devices are useful for historical analysis, but they may not provide real-time web-based alerts or centralized access. If the data is reviewed only after an incident occurs, corrective action may be delayed.

Modern monitoring systems use IoT sensors, cloud platforms, and dashboards. However, such solutions may be expensive or complex for academic, small-scale, or prototype environments. This project provides a software-based simulation of such a system, allowing the logic of monitoring, alerting, and dashboard analytics to be developed and tested without physical hardware.

2.2 Limitations of Traditional Systems

The limitations of traditional or manual cold storage monitoring methods include:

- **Delayed Detection:** Manual checks cannot detect changes immediately.
- **Human Error:** Readings may be recorded incorrectly or skipped.
- **No Real-Time Alerting:** Manual systems usually do not generate automatic alerts.

- **Limited Accountability:** It is difficult to track who handled or resolved incidents.
- **Poor Data Visualization:** Registers and spreadsheets do not provide real-time dashboards.
- **Lack of Role Control:** Traditional systems may not separate admin and user responsibilities.
- **No Automated Testing:** Without simulation, testing abnormal conditions is difficult.

These limitations show the need for a centralized software platform that supports automated data generation, rule-based alerting, user-specific access, and dashboard visualization.

2.3 Proposed Improvement

The proposed Cold Storage Monitoring System improves upon traditional methods by introducing a web-based backend and frontend application. The system allows role-based access for admins and users. It automatically creates sensors when storage units are created and includes a simulator that continuously generates readings.

The proposed system provides the following improvements:

- Centralized management of warehouses, storage units, sensors, and alerts.
- Automated sensor data generation for testing and demonstration.
- Rule-based alert generation for abnormal sensor values.
- Admin-level visibility of users, warehouses, alert counts, and charts.
- User-level visibility limited to assigned warehouses and own alerts.
- Dashboard charts for alert type distribution and storage-unit-wise alert counts.
- Dependency cleanup during deletion of warehouses, storage units, and sensors.

The proposed system is flexible and can be extended to support real sensors, cloud databases, authentication tokens, and notification services.

3 System Analysis

3.1 Functional Requirements

Functional requirements define the operations that the system must perform. The functional requirements of the Cold Storage Monitoring System are divided according to user roles and system modules.

3.1.1 Admin Functional Requirements

1. Admin must be able to log in using admin credentials.
2. Admin must be able to view registered users.
3. Admin must be able to create or assign warehouses to users.
4. Admin must be able to view the total number of warehouses.
5. Admin must be able to view total alerts, active alerts, and solved alerts.
6. Admin must be able to view alert-related visual analytics.
7. Admin must be able to view a pie chart of alert types.
8. Admin must be able to view a bar graph of alerts per storage unit.
9. Admin must be able to view the list of warehouses.
10. Admin must be able to delete warehouses with dependency cleanup.
11. Admin must be able to view alerts but must not be able to resolve them.

3.1.2 User Functional Requirements

1. User must be able to register.

2. User must be able to log in.
3. User must be able to view only warehouses assigned to that user.
4. User must be able to create storage units under assigned warehouses.
5. Default sensors must be created automatically when a storage unit is created.
6. User must be able to view their own storage units and sensors.
7. User must be able to delete their own storage units and sensors.
8. User must be able to view alerts related only to their own sensors and storage units.
9. User must be able to resolve only their own alerts.

3.1.3 Simulator Functional Requirements

1. Simulator must fetch active sensors from the backend or database.
2. Simulator must randomly select sensors.
3. Simulator must generate sensor readings.
4. Simulator must generate both normal and abnormal values.
5. Simulator must submit generated readings to the backend `/data/` endpoint.
6. Simulator must run automatically along with the FastAPI backend.
7. Simulator must trigger alerts by generating abnormal readings.

3.1.4 Alert Functional Requirements

1. System must generate alerts when temperature exceeds maximum threshold.
2. System must generate alerts when humidity exceeds maximum threshold.
3. System must generate alerts when door status is open.
4. Alerts must be stored in the database.
5. Alerts must be displayed on dashboards.
6. Alert status must support active and solved states.
7. Users must be able to resolve only their own alerts.

3.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes of the system.

Table 3.1: Non-Functional Requirements

Requirement	Description
Usability	The system should provide simple web pages for registration, login, dashboards, warehouse management, storage unit management, and alert viewing.
Performance	The backend should respond quickly to dashboard and API requests under normal academic project usage.
Maintainability	The code should be modular, separating routes, models, database logic, templates, and simulator logic.
Security	Access must be role-based so that users cannot access warehouses or alerts belonging to others.
Reliability	Sensor readings and alerts must be stored consistently in the database.
Scalability	The architecture should allow future migration from SQLite to a production database such as PostgreSQL.
Extensibility	The simulator should be replaceable with real IoT sensor input in the future.
Portability	The system should run on a local machine using Python, FastAPI, SQLite, and Uvicorn.

3.3 Feasibility Analysis

3.3.1 Technical Feasibility

The project is technically feasible because it uses widely available open-source technologies. FastAPI provides a modern and efficient backend framework for building APIs and web applications. SQLAlchemy simplifies database interaction and object-relational mapping. SQLite is suitable for local development and academic demonstration. HTML, CSS, and Jinja2 templates allow server-side rendering of frontend pages without requiring a complex JavaScript framework.

The simulator can be implemented using Python and the Requests library. Since the backend exposes a data endpoint, the simulator can send generated sensor values through HTTP requests.

3.3.2 Operational Feasibility

The system is operationally feasible because it provides user-friendly dashboards for both administrators and users. Administrators can manage warehouses and monitor overall alert statistics, while users can manage their own storage units and resolve their alerts. The role separation makes the workflow easy to understand.

3.3.3 Economic Feasibility

The project is economically feasible because it uses free and open-source tools. No paid cloud services or hardware devices are required for the initial version. The simulator removes the need for physical sensors during development and testing.

3.3.4 Schedule Feasibility

The project can be developed within an academic project timeline because the scope is clearly defined. Core features such as authentication, CRUD operations, sensor simulation, alert generation, and dashboards can be implemented incrementally.

4 System Design

4.1 Overall Architecture

The system follows a layered architecture. The presentation layer contains HTML, CSS, and Jinja2 templates. The application layer is implemented using FastAPI routes and controllers. The business logic layer includes role checking, warehouse management, sensor management, simulator integration, and alert generation. The persistence layer uses SQLAlchemy models and SQLite database.

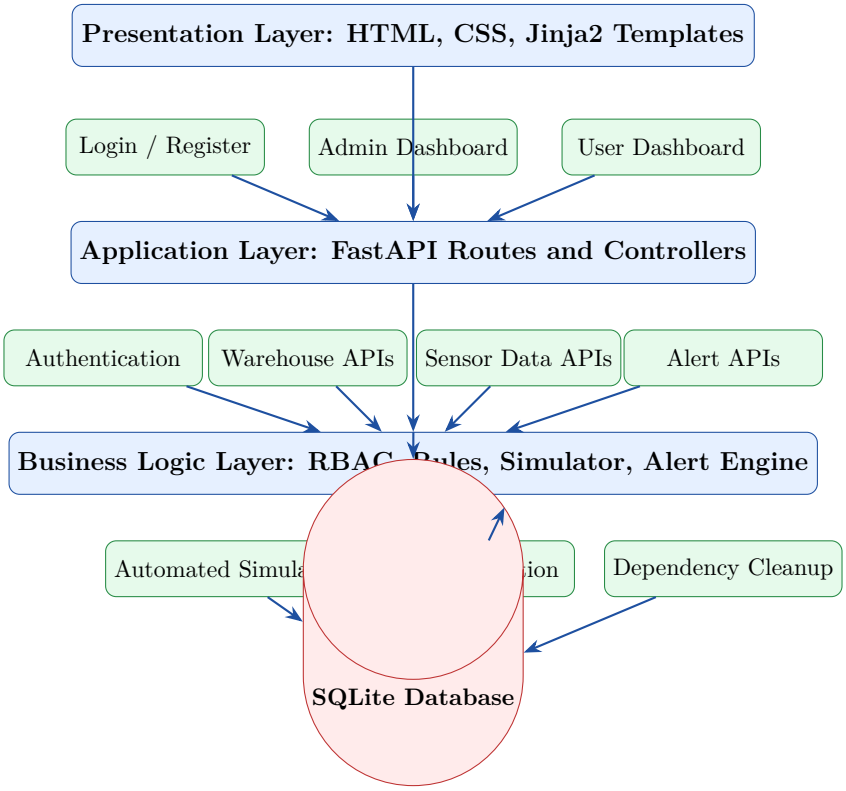

Figure 4.1: System Architecture Diagram

4.2 Module Description

4.2.1 Authentication Module

The authentication module handles user registration, login, role identification, and session-style access using cookies. When a user logs in, the system identifies whether the user is an Admin or a User. Based on the role, the system redirects the user to the appropriate dashboard.

4.2.2 Admin Module

The admin module provides administrative control over registered users, warehouse assignment, warehouse deletion, alert visibility, and dashboard analytics. Admin can view all alerts but cannot resolve them. This design ensures that alert resolution remains the responsibility of the user who owns the storage unit.

4.2.3 User Module

The user module provides user-specific access to assigned warehouses, storage units, sensors, and alerts. A user can create storage units under assigned warehouses. Whenever a storage unit is created, the system automatically creates three default sensors: temperature sensor, humidity sensor, and door status sensor.

4.2.4 Warehouse Management Module

The warehouse management module allows administrators to assign warehouses to users. Users can only view warehouses assigned to them. Warehouse deletion includes dependency cleanup so that related storage units, sensors, sensor data, and alerts are handled consistently.

4.2.5 Storage Unit Management Module

The storage unit module allows users to create and manage storage units. A storage unit represents a physical compartment such as a cold room, freezer, refrigerator, or vaccine chamber. Each storage unit contains sensors and can generate alerts.

4.2.6 Sensor Management Module

The sensor module maintains records of sensors associated with storage units. The system uses three main sensor types: temperature, humidity, and door status. Sensors are created

automatically to reduce manual configuration effort.

4.2.7 Sensor Data Module

The sensor data module stores readings generated by the simulator. Each reading is associated with a sensor and includes the sensor value and timestamp. Sensor data is used by the alert engine to determine whether an abnormal condition has occurred.

4.2.8 Alert Management Module

The alert module stores alerts generated by rule violations. Alerts include type, status, severity or message, related sensor, related storage unit, and timestamps. Users can resolve alerts that belong to their own storage units.

4.2.9 Simulator Module

The simulator module automatically generates sensor data. It fetches active sensors, randomly selects sensors, produces normal and abnormal values, and submits the generated readings to the backend. The abnormal values are intentionally generated at intervals so that the alert system can be tested.

4.3 User Role Design

The system uses role-based access control with two main roles: Admin and User. The role design ensures that users can access only the data relevant to them.

Table 4.1: Admin and User Role Comparison

Feature	Admin	User
Login	Yes	Yes
Registration	Usually pre-created admin	Can register
View registered users	Yes	No
Assign warehouses	Yes	No
View warehouse count	Yes	Own assigned warehouses only
Create storage units	No	Yes, under assigned warehouses
Automatic sensor creation	Indirect	Yes, triggered by storage unit creation
View all alerts	Yes	No, own alerts only
Resolve alerts	No	Yes, own alerts only
View charts	Yes	Limited/user-specific if implemented
Delete warehouses	Yes	No
Delete own storage units	No	Yes
Delete own sensors	No	Yes

4.4 Database Design

The database is designed using SQLAlchemy models and SQLite. The main entities are User, Warehouse, StorageUnit, Sensor, SensorData, Alert, AlertLog, and Rule.

Table 4.2: Main Database Entities

Entity	Description
User	Stores user details, login credentials, and role information.
Warehouse	Stores warehouse details and ownership/assignment information.
StorageUnit	Stores cold room or unit details under a warehouse.
Sensor	Stores sensor details such as type and active status.
SensorData	Stores generated readings from sensors.
Alert	Stores alerts generated due to rule violations.
AlertLog	Stores alert history or action logs such as resolution.
Rule	Stores rule definitions and threshold conditions.

4.5 ER Model Explanation

The ER model defines how the main entities are related. A user may have one or more assigned warehouses. A warehouse may contain multiple storage units. A storage unit automatically contains multiple sensors. Each sensor may generate many sensor data records. Based on sensor data and rule evaluation, alerts may be created. Alert logs maintain historical actions related to alerts.

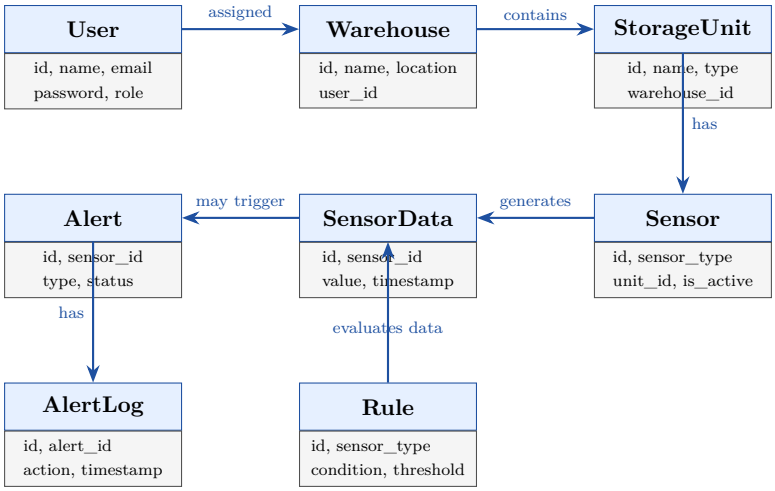

Figure 4.2: Entity Relationship Diagram

4.6 Flow of Simulator and Alert Generation

The simulator and alert generation workflow is one of the most important parts of the system. The simulator runs automatically and repeatedly performs sensor selection and data generation. Once the backend receives data, it checks the sensor type and applies the corresponding rule.

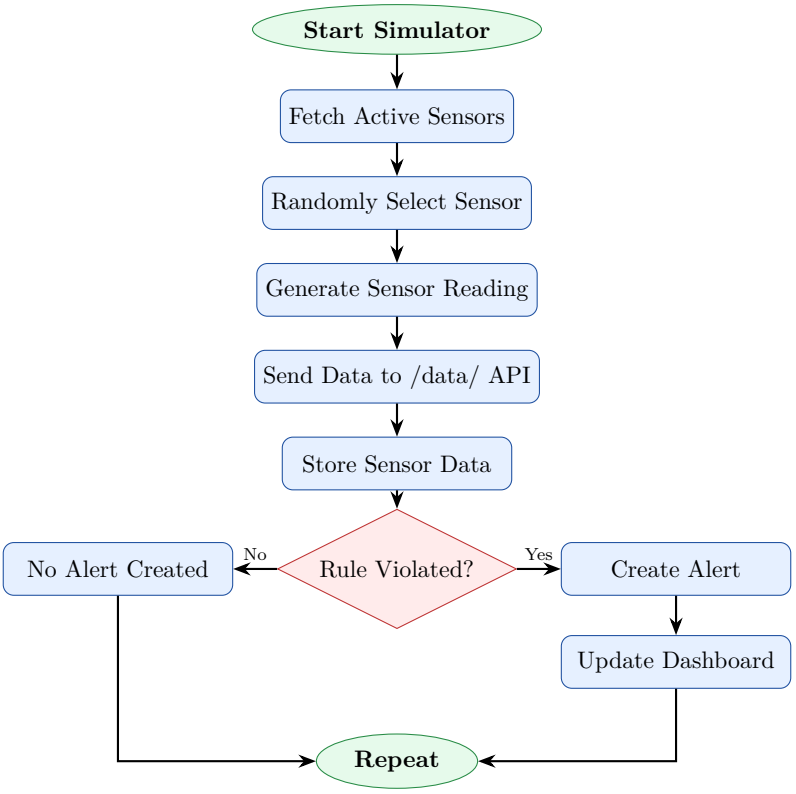

Figure 4.3: Simulator and Alert Generation Flow Diagram

4.7 User Interface Figure Placeholders

Placeholder for Figure

Login Page

Insert screenshot or diagram here during final documentation.

Figure 4.4: Login page for Admin and User authentication

Placeholder for Figure

Register Page

Insert screenshot or diagram here during final documentation.

Figure 4.5: Register page for new user registration

Placeholder for Figure

Admin Dashboard

Insert screenshot or diagram here during final documentation.

Figure 4.6: Admin dashboard showing counts, users, warehouses, charts, and alerts

Placeholder for Figure

Warehouse Assignment Page

Insert screenshot or diagram here during final documentation.

Figure 4.7: Warehouse assignment page used by admin to assign warehouses to users

Placeholder for Figure

Storage Unit Creation Page

Insert screenshot or diagram here during final documentation.

Figure 4.8: Storage unit creation interface for assigned warehouses

Placeholder for Figure

Sensor List Page

Insert screenshot or diagram here during final documentation.

Figure 4.9: Sensor list showing automatically created temperature, humidity, and door status sensors

Placeholder for Figure

Alerts Table

Insert screenshot or diagram here during final documentation.

Figure 4.10: Alerts table showing active and solved alerts

Placeholder for Figure

Pie Chart

Insert screenshot or diagram here during final documentation.

Figure 4.11: Pie chart showing alert type distribution

Placeholder for Figure

Bar Graph

Insert screenshot or diagram here during final documentation.

Figure 4.12: Bar graph showing alerts per storage unit

Placeholder for Figure

Login Dashboard

Insert screenshot or diagram here during final documentation.

Figure 4.13: Login dashboard or landing screen of the application

5 Implementation

5.1 Backend Implementation Using FastAPI

The backend of the system is developed using FastAPI. FastAPI is used to define routes for authentication, user registration, warehouse management, storage unit management, sensor management, sensor data submission, alert viewing, and dashboard rendering.

The backend performs the following responsibilities:

- Receives form submissions from Jinja2 templates.
- Handles registration and login.
- Maintains role-based access control.
- Performs database operations using SQLAlchemy sessions.
- Creates warehouses, storage units, sensors, sensor data, and alerts.
- Provides endpoints for simulator-generated data.
- Renders dashboards with dynamic values.

A simplified backend route pattern is shown below.

Listing 5.1: Illustrative FastAPI Route Structure

```
from fastapi import FastAPI, Request, Depends
from fastapi.responses import HTMLResponse
from sqlalchemy.orm import Session

app = FastAPI(title="Cold Storage Monitoring System")

@app.get("/")
def home():
```

```
    return {"message": "Cold Storage Monitoring System is running"}

@app.post("/data/")
def receive_sensor_data(payload: dict, db: Session = Depends(get_db)):
    # Store sensor data
    # Evaluate rule
    # Generate alert if required
    return {"status": "data received"}
```

5.2 Frontend Implementation Using HTML, CSS, and Jinja2

The frontend is implemented using HTML and CSS with Jinja2 templates. Jinja2 allows dynamic values to be inserted into web pages from the backend. For example, the admin dashboard can display total users, warehouse count, alert count, and chart data fetched from the database.

The frontend includes pages such as:

- Login page.
- Register page.
- Admin dashboard.
- User dashboard.
- Warehouse assignment page.
- Storage unit creation page.
- Sensor list page.
- Alerts page.

The visual dashboard includes a pie chart for alert type distribution and a bar graph for alerts per storage unit. These charts can be implemented using frontend chart libraries such as Chart.js or by rendering chart data into templates.

5.3 Authentication System

The authentication system supports session-style login using cookies. When a user logs in, the backend validates credentials and identifies the user role. The role is then used to control access to different pages and operations.

The two main roles are:

- **Admin:** Can view users, assign warehouses, delete warehouses, view charts, and view alerts.
- **User:** Can view assigned warehouses, create storage units, manage own sensors, view own alerts, and resolve own alerts.

Role-based restrictions are important because a normal user must not be able to access another user's warehouse or resolve another user's alert.

5.4 Warehouse and Storage Unit Management

Warehouse management is mainly performed by the administrator. The admin can create or assign warehouses to users. Each warehouse belongs to a user assignment. The user can only view warehouses assigned to them.

Storage unit management is performed by the user. A storage unit is created under an assigned warehouse. Once a storage unit is created, the system automatically creates default sensors for that storage unit. This reduces manual work and ensures that every storage unit is ready for monitoring.

5.5 Sensor and Alert Management

Each storage unit has default sensors:

1. Temperature sensor.
2. Humidity sensor.
3. Door status sensor.

The sensor data generated by the simulator is stored in the SensorData table. The alert engine evaluates the data against rules. If the value violates a threshold, an alert is created.

Common alert types include:

- Temperature High Alert.
- Humidity High Alert.
- Door Open Alert.

Users can resolve only their own alerts. Admin can view alerts for supervision but cannot resolve them.

5.6 Automatic Sensor Generation

Automatic sensor generation is a key feature of this project. When a user creates a storage unit, the backend immediately creates three sensor records. This ensures consistency and prevents incomplete configuration.

The logic can be described as follows:

Listing 5.2: Illustrative Automatic Sensor Creation Logic

```
def create_default_sensors(storage_unit_id, db):
    sensor_types = ["temperature", "humidity", "door_status"]

    for sensor_type in sensor_types:
        sensor = Sensor(
            storage_unit_id=storage_unit_id,
            sensor_type=sensor_type,
            is_active=True
        )
        db.add(sensor)

    db.commit()
```

5.7 Simulator Integration

The simulator is integrated with the backend and runs automatically. It uses the Requests library to send generated readings to the backend endpoint. The simulator may run in a background thread or separate process depending on implementation.

The simulator performs these steps:

1. Fetch active sensors.
2. Randomly select a sensor.
3. Generate a value based on sensor type.
4. Sometimes generate abnormal values.
5. Send the generated value to the `/data/` endpoint.
6. Repeat the process continuously.

Listing 5.3: Illustrative Simulator Logic

```python
import random
import time
import requests

BASE_URL = "http://127.0.0.1:8000"

def generate_value(sensor_type):
    if sensor_type == "temperature":
        return random.choice([random.uniform(2, 8), random.uniform(9, 15)])
    if sensor_type == "humidity":
        return random.choice([random.uniform(40, 60), random.uniform(65, 85)])
    if sensor_type == "door_status":
        return random.choice(["closed", "open"])
    return None

def simulator_loop():
    while True:
        sensors = requests.get(f"{BASE_URL}/sensors/active").json()
        if sensors:
            sensor = random.choice(sensors)
            value = generate_value(sensor["sensor_type"])
            payload = {
                "sensor_id": sensor["id"],
                "value": value
            }
            requests.post(f"{BASE_URL}/data/", json=payload)
        time.sleep(5)
```

5.8 Alert Rule Implementation

Alert rules are based on sensor type and threshold conditions. For example, if a temperature reading is greater than the maximum allowed temperature, a temperature alert is generated. If the humidity value exceeds the maximum threshold, a humidity alert is generated. If the door status is open, a door alert is generated.

Table 5.1: Alert Generation Rules

Sensor Type	**Condition**	**Alert Generated**
Temperature	Value exceeds maximum threshold	Temperature High Alert
Humidity	Value exceeds maximum threshold	Humidity High Alert
Door Status	Door value is open	Door Open Alert

6 Results and Discussion

6.1 Project Screenshots Description

This section describes the expected screenshots of the completed project. The actual screenshots can be inserted in the placeholder figures given in the System Design chapter.

6.1.1 Login Page

The login page allows Admin and User accounts to enter credentials. After successful authentication, the system redirects the user to the appropriate dashboard based on role.

6.1.2 Register Page

The register page allows a new user to create an account. After registration, the user can log in and access warehouses assigned by the administrator.

6.1.3 Admin Dashboard

The admin dashboard displays registered users, warehouse count, total alerts, active alerts, solved alerts, alert type distribution pie chart, alerts per storage unit bar graph, and warehouse list. The admin can delete warehouses but cannot resolve alerts.

6.1.4 Warehouse Assignment Page

The warehouse assignment page allows the admin to create or assign warehouses for registered users. This ensures that each user can access only their assigned warehouses.

6.1.5 Storage Unit Creation Page

The storage unit creation page allows users to create storage units under assigned warehouses. After creation, default sensors are automatically generated.

6.1.6 Sensor List Page

The sensor list page displays sensors belonging to a user's storage units. It includes temperature, humidity, and door status sensors.

6.1.7 Alerts Table

The alerts table displays active and solved alerts. Users can resolve only their own alerts. Admin can view alerts but cannot resolve them.

6.1.8 Pie Chart and Bar Graph

The pie chart shows the distribution of alert types such as temperature, humidity, and door alerts. The bar graph shows the number of alerts per storage unit.

6.2 Dashboard Analytics Explanation

Dashboard analytics help users and administrators understand the current state of the system. The admin dashboard provides a global view, while the user dashboard provides a user-specific view.

Important dashboard metrics include:

- Total warehouses.
- Total alerts.
- Active alerts.
- Solved alerts.
- Alert type distribution.
- Alerts per storage unit.
- Number of registered users.

These analytics improve decision-making by showing where most alerts occur and which storage units need attention.

6.3 User and Admin Feature Comparison

Table 6.1: Feature Comparison Between Admin and User

Feature	**Admin**	**User**
Login	Yes	Yes
Register	No / Optional	Yes
View users	Yes	No
Assign warehouses	Yes	No
View assigned warehouses	All / Assigned view	Own assigned only
Create storage units	No	Yes
View sensors	All / Dashboard level	Own sensors only
Delete sensors	No	Own sensors only
View alerts	All alerts	Own alerts only
Resolve alerts	No	Own alerts only
View charts	Yes	Optional / own data
Delete warehouses	Yes	No

6.4 Test Observations

During testing, the simulator successfully generated sensor data for active sensors. Normal values were stored without alerts, while abnormal values generated alerts based on the defined rules. The automatic sensor creation feature reduced manual effort and ensured that every storage unit had the required sensors.

The role-based access control worked as expected. Admin users were able to view users and warehouse statistics, while normal users could access only their assigned warehouses and own alerts. The alert resolution restriction ensured that users could not resolve alerts belonging to other users.

7 Testing

7.1 Unit Testing

Unit testing focuses on individual components such as sensor value generation, rule evaluation, automatic sensor creation, and alert creation. These tests verify whether each function behaves correctly in isolation.

Examples of unit testing areas include:

- Testing whether default sensors are created for a new storage unit.
- Testing whether high temperature generates an alert.
- Testing whether high humidity generates an alert.
- Testing whether door open status generates an alert.
- Testing whether normal values do not generate alerts.

7.2 Functional Testing

Functional testing verifies the complete workflow of the system. It ensures that the application works correctly from the user's perspective.

Functional testing includes:

- User registration and login.
- Admin login.
- Warehouse assignment.
- Storage unit creation.
- Sensor list display.

- Simulator data submission.
- Alert display.
- Alert resolution.
- Dashboard chart rendering.

7.3 Test Cases

Table 7.1: Sample Test Cases

ID	Test Case	Input / Action	Expected Result	Status
TC01	User registration	Enter valid name, email, password	User account is created	Pass
TC02	User login	Enter valid user credentials	User dashboard opens	Pass
TC03	Admin login	Enter valid admin credentials	Admin dashboard opens	Pass
TC04	Invalid login	Enter wrong password	Error message is displayed	Pass
TC05	Warehouse assignment	Admin assigns warehouse to user	Warehouse appears in user's dashboard	Pass
TC06	Storage unit creation	User creates storage unit	Storage unit is saved	Pass
TC07	Default sensor creation	Create storage unit	Temperature, humidity, and door sensors are created	Pass
TC08	Normal temperature data	Submit temperature within range	Data stored without alert	Pass
TC09	High temperature data	Submit temperature above threshold	Temperature alert generated	Pass
TC10	High humidity data	Submit humidity above threshold	Humidity alert generated	Pass
TC11	Door open data	Submit door status as open	Door open alert generated	Pass
TC12	User alert resolution	User resolves own alert	Alert status changes to solved	Pass
TC13	Unauthorized alert resolution	User tries to resolve another user's alert	Access denied or operation blocked	Pass

ID	Test Case	Input / Action	Expected Result	Status
TC14	Admin alert resolution restriction	Admin attempts to resolve alert	Resolution not allowed	Pass
TC15	Warehouse deletion	Admin deletes warehouse	Related dependencies are cleaned up	Pass
TC16	Chart display	Admin opens dashboard	Pie chart and bar graph are displayed	Pass

7.4 Testing Summary

The testing phase confirmed that the project meets its functional requirements. The simulator successfully generated sensor readings, abnormal data triggered alerts, role-based access restrictions were enforced, and dashboards displayed expected values. The system is suitable for academic demonstration and can be further improved for production usage.

8 Advantages, Limitations, and Future Enhancements

8.1 Advantages

The Cold Storage Monitoring System provides several advantages:

- Provides centralized monitoring of warehouses, storage units, sensors, and alerts.
- Supports role-based access control for Admin and User roles.
- Automatically creates default sensors for storage units.
- Includes a simulator for automated sensor data generation.
- Generates alerts based on threshold violations.
- Provides dashboard analytics using pie charts and bar graphs.
- Allows users to resolve their own alerts.
- Maintains historical data in a database.
- Uses open-source technologies and is cost-effective.
- Can be extended to support real IoT sensors in the future.

8.2 Limitations

Although the system provides many useful features, it also has some limitations:

- The current version uses simulated sensor data instead of physical IoT devices.
- SQLite is suitable for local development but may not be ideal for large-scale deployment.

- Authentication is based on cookie/session-style logic and can be improved using JWT or secure server-side sessions.
- The system does not currently include email, SMS, or push notifications.
- The dashboard depends on stored data and does not provide real-time WebSocket updates.
- Advanced audit compliance formats are not included in the current version.
- Machine learning based anomaly prediction is not implemented.

8.3 Future Enhancements

The project can be enhanced in several ways:

1. Integrate real IoT sensors using MQTT, HTTP, or serial communication.
2. Replace SQLite with PostgreSQL or MySQL for production deployment.
3. Implement JWT-based authentication and stronger password hashing.
4. Add email and SMS notifications for critical alerts.
5. Implement real-time dashboards using WebSockets.
6. Add role levels such as Auditor, Supervisor, and Technician.
7. Export compliance reports as PDF or Excel files.
8. Add machine learning models to predict storage failures.
9. Deploy the application on cloud platforms such as Azure, AWS, or GCP.
10. Add Docker support for easier deployment.

9 Conclusion

The Cold Storage Monitoring System is a complete web-based software project designed to monitor environmental conditions in cold storage environments. The system manages users, warehouses, storage units, sensors, sensor data, rules, alerts, and alert logs. It implements role-based access control with Admin and User roles, ensuring that each role has appropriate permissions.

The project successfully demonstrates automatic sensor creation, automated sensor data simulation, rule-based alert generation, dashboard analytics, and user-specific alert resolution. The admin dashboard provides overall visibility into users, warehouses, alert counts, and charts, while the user dashboard provides access to assigned warehouses, storage units, sensors, and own alerts.

The use of FastAPI, SQLAlchemy, SQLite, HTML, CSS, and Jinja2 makes the system modular, understandable, and suitable for academic submission. The simulator allows the complete monitoring workflow to be tested without physical hardware. Although the current version uses simulated data, the architecture can be extended to integrate real IoT devices and production-level deployment features.

Overall, the project provides a strong foundation for real-world cold storage monitoring and demonstrates practical knowledge of backend development, database design, role-based access control, automated simulation, alert processing, and dashboard-based visualization.

Bibliography

[1] FastAPI Documentation, *FastAPI Framework for Building APIs with Python*, Available at: https://fastapi.tiangolo.com/

[2] SQLAlchemy Documentation, *The Python SQL Toolkit and Object Relational Mapper*, Available at: https://docs.sqlalchemy.org/

[3] SQLite Documentation, *SQLite Database Engine*, Available at: https://www.sqlite.org/docs.html

[4] Jinja Documentation, *Jinja Template Engine*, Available at: https://jinja.palletsprojects.com/

[5] Uvicorn Documentation, *ASGI Web Server Implementation for Python*, Available at: https://www.uvicorn.org/

[6] Python Software Foundation, *Python Programming Language Documentation*, Available at: https://docs.python.org/3/

[7] Chart.js Documentation, *Simple Yet Flexible JavaScript Charting Library*, Available at: https://www.chartjs.org/docs/

[8] Requests Documentation, *HTTP Library for Python*, Available at: https://requests.readthedocs.io/

A Sample API Endpoints

A.1 API Endpoint Summary

Table A.1: Sample API Endpoints

Method	Endpoint	Description
GET	`/`	Home or health check endpoint
GET	`/login`	Displays login page
POST	`/login`	Authenticates user or admin
GET	`/register`	Displays user registration page
POST	`/register`	Creates a new user account
GET	`/admin/dashboard`	Displays admin dashboard
GET	`/user/dashboard`	Displays user dashboard
POST	`/warehouses/create`	Creates or assigns warehouse
GET	`/warehouses`	Displays warehouse list
POST	`/warehouses/delete/{id}`	Deletes warehouse and related dependencies
POST	`/storage-units/create`	Creates storage unit and default sensors
GET	`/storage-units`	Displays storage units
GET	`/sensors`	Displays sensors
GET	`/sensors/active`	Returns active sensors for simulator
POST	`/data/`	Receives simulator-generated sensor data
GET	`/alerts`	Displays alerts
POST	`/alerts/resolve/{id}`	Resolves user-owned alert
GET	`/logout`	Logs out current user

A.2 Sample Data Submission Payload

Listing A.1: Sample Sensor Data Payload

```
{
    "sensor_id": 12,
    "value": 10.5,
    "timestamp": "2026-06-10T10:30:00"
}
```

B Sample Database Tables

B.1 User Table

Table B.1: Sample User Table

Column	Type	Description
id	Integer	Primary key
name	String	Full name of user
email	String	Login email
password	String	Hashed or stored password depending on implementation
role	String	Admin or User
created_at	DateTime	Account creation time

B.2 Warehouse Table

Table B.2: Sample Warehouse Table

Column	Type	Description
id	Integer	Primary key
name	String	Warehouse name
location	String	Warehouse location
user_id	Integer	Assigned user reference
created_at	DateTime	Creation timestamp

B.3 Storage Unit Table

Table B.3: Sample Storage Unit Table

Column	Type	Description
id	Integer	Primary key
name	String	Storage unit name
unit_type	String	Freezer, cold room, refrigerator, etc.
warehouse_id	Integer	Related warehouse reference
created_at	DateTime	Creation timestamp

B.4 Sensor Table

Table B.4: Sample Sensor Table

Column	Type	Description
id	Integer	Primary key
sensor_type	String	Temperature, humidity, or door status
storage_unit_id	Integer	Related storage unit reference
is_active	Boolean	Indicates whether simulator can use the sensor
created_at	DateTime	Creation timestamp

B.5 Sensor Data Table

Table B.5: Sample Sensor Data Table

Column	Type	Description
id	Integer	Primary key
sensor_id	Integer	Related sensor reference
value	String / Float	Sensor reading value
timestamp	DateTime	Reading time

B.6 Alert Table

Table B.6: Sample Alert Table

Column	Type	Description
id	Integer	Primary key
sensor_id	Integer	Related sensor reference
storage_unit_id	Integer	Related storage unit reference
alert_type	String	Temperature, humidity, or door alert
message	String	Alert description
status	String	Active or solved
created_at	DateTime	Alert creation time
resolved_at	DateTime	Alert resolution time

C Sample Simulator Flow

C.1 Simulator Flow Description

The simulator is responsible for generating sensor data automatically. It continuously checks for active sensors and selects one at random. Based on the selected sensor type, it generates a suitable value. For temperature sensors, it may generate values within the safe range or above the maximum threshold. For humidity sensors, it may generate normal or high humidity values. For door status sensors, it may generate open or closed values.

After generating the value, the simulator sends the data to the backend `/data/` endpoint. The backend stores the data and evaluates it against alert rules. If a violation is detected, an alert is created and shown on the dashboard.

C.2 Simulator Pseudocode

Listing C.1: Simulator Pseudocode

```
START
    WHILE application is running
        FETCH active sensors
        IF active sensors exist THEN
            SELECT one sensor randomly
            IF sensor type is temperature THEN
                GENERATE normal or high temperature value
            ELSE IF sensor type is humidity THEN
                GENERATE normal or high humidity value
            ELSE IF sensor type is door status THEN
                GENERATE open or closed value
            END IF

            SEND generated reading to /data/ endpoint
```

```
            BACKEND stores reading
            BACKEND checks rule
            IF rule is violated THEN
                CREATE alert
            END IF
        END IF
        WAIT for configured interval
    END WHILE
END
```

C.3 Sample Rule Evaluation Logic

Listing C.2: Illustrative Rule Evaluation Logic

```python
def evaluate_sensor_data(sensor, value):
    if sensor.sensor_type == "temperature":
        if float(value) > 8:
            return "Temperature exceeds maximum threshold"

    elif sensor.sensor_type == "humidity":
        if float(value) > 60:
            return "Humidity exceeds maximum threshold"

    elif sensor.sensor_type == "door_status":
        if str(value).lower() == "open":
            return "Door is open"

    return None
```

D Glossary

Table D.1: Glossary of Terms

Term	**Meaning**
Admin	User role responsible for managing users, warehouses, and viewing system-wide analytics.
User	User role responsible for managing assigned warehouses, storage units, sensors, and own alerts.
Warehouse	A cold storage facility assigned to a user.
Storage Unit	A specific cold room, freezer, or refrigerator under a warehouse.
Sensor	A logical monitoring device for temperature, humidity, or door status.
Sensor Data	Reading generated by a sensor and stored in the database.
Alert	Notification created when a sensor reading violates a rule.
Rule	Condition used to evaluate sensor readings.
Simulator	Automated module that generates virtual sensor readings.
RBAC	Role-Based Access Control, used to restrict access based on user role.
FastAPI	Python web framework used for backend development.
SQLAlchemy	ORM used to interact with the database.
SQLite	Lightweight relational database used for local storage.
Jinja2	Template engine used to render dynamic HTML pages.